

Rapport BIG DATA - Rendu

1. Le dataset choisi

J'ai pris les données DVF sur data.gouv.fr, ça recense toutes les transactions immobilières en France. Je me suis limité au département 78 pour garder un volume raisonnable, 54k lignes avec date, type de bien, prix, surface, nombre de pièces, commune et coordonnées GPS.

2. Les traitements réalisés

Le fichier n'était pas utilisable directement, donc j'ai d'abord fait pas mal de nettoyage. J'ai gardé seulement les ventes, puis seulement les maisons et les appartements. J'ai ensuite enlevé les doublons et les lignes où il manquait le prix ou la surface.

J'ai ensuite calculé le prix au m^2 de chaque vente et supprimé les prix trop bas ou trop hauts, sûrement des erreurs. Il reste au final 16 126 ventes sur les 54 134 lignes de départ.

Pour l'analyse, je suis restée sur ces choses :

- les statistiques de base sur le prix, la surface et le nombre de pièces : moyenne, médiane, minimum et maximum
- le lien entre la surface et le prix
- le prix moyen au m^2 dans chaque commune

3. Les résultats observés

Le prix moyen d'une vente est d'environ 394 000 €, contre 295 000 € pour la médiane. La différence vient de quelques ventes très chères, jusqu'à 9 M€.

La corrélation entre la surface et le prix est de **0,44** : la surface joue sur le prix, mais l'emplacement compte aussi beaucoup.

Entre Versailles et Élancourt, le prix au m^2 varie presque du simple au triple. Versailles et Saint-Germain arrivent en tête, ce qui correspond à leur réputation.

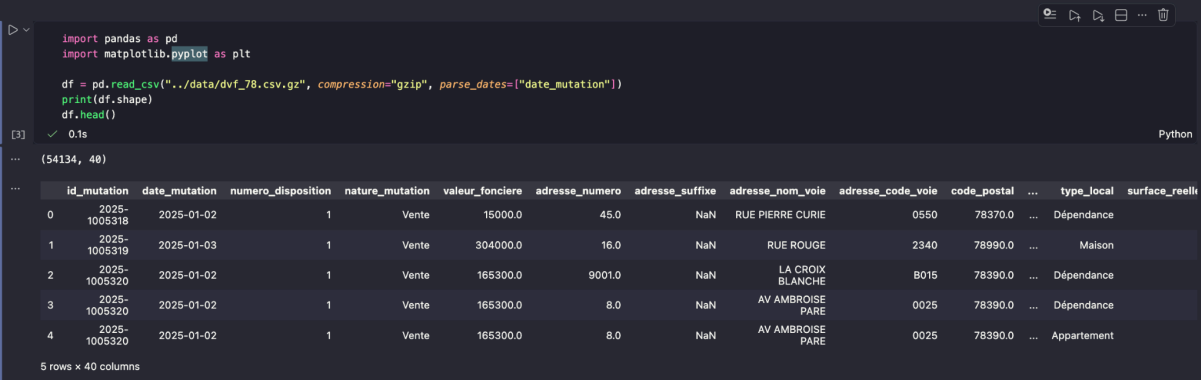
Le graphique est dans le fichier : ["screen/prix_m2_par_commune.png"](#)

4. Les limites et pistes d'amélioration

La principale limite, c'est que je n'ai travaillé que sur un seul département. Sur la France entière, le volume serait beaucoup plus important et un seul ordinateur ne suffirait probablement plus : il faudrait passer sur un outil comme Spark pour répartir les calculs sur un cluster.

Captures pour le livrable ci-dessous

Chargement :



```
import pandas as pd
import matplotlib.pyplot as plt

df = pd.read_csv("../data/dvf_78.csv.gz", compression="gzip", parse_dates=["date_mutation"])
print(df.shape)
df.head()
```

[3] ✓ 0.1s Python

(54134, 40)

	id_mutation	date_mutation	numero_disposition	nature_mutation	valeur_fonciere	adresse_numero	adresse_suffixe	adresse_nom_voie	adresse_code_voie	code_postal	...	type_local	surface_reelle
0	2025-1005318	2025-01-02	1	Vente	15000.0	45.0	NaN	RUE PIERRE CURIE	0550	78370.0	...	Dépendance	
1	2025-1005319	2025-01-03	1	Vente	304000.0	16.0	NaN	RUE ROUGE	2340	78990.0	...	Maison	
2	2025-1005320	2025-01-02	1	Vente	165300.0	9001.0	NaN	LA CROIX BLANCHE	B015	78390.0	...	Dépendance	
3	2025-1005320	2025-01-02	1	Vente	165300.0	8.0	NaN	AV AMBROISE PARE	0025	78390.0	...	Dépendance	
4	2025-1005320	2025-01-02	1	Vente	165300.0	8.0	NaN	AV AMBROISE PARE	0025	78390.0	...	Appartement	

5 rows x 40 columns

Traitement :

Traitement

```
cols = ["date_mutation", "nature_mutation", "valeur_fonciere", "nom_commune", "code_postal",
        "type_local", "surface_reelle_bati", "nombre_pieces_principales", "longitude", "latitude"]
df = df[cols]

df = df[df["nature_mutation"] == "Vente"]
df = df[df["type_local"].isin(["Maison", "Appartement"])]

df = df.drop_duplicates(subset=["valeur_fonciere", "surface_reelle_bati"])
df = df[df[["valeur_fonciere"] > 0] & (df[["surface_reelle_bati"] > 0)]

df["prix_m2"] = (df["valeur_fonciere"] / df["surface_reelle_bati"]).round(2)
df = df[df["prix_m2"].between(df["prix_m2"].quantile(0.01), df["prix_m2"].quantile(0.99))]

print(df.shape)
df.head()
```

[4] ✓ 0.0s Python

(16126, 11)

	date_mutation	nature_mutation	valeur_fonciere	nom_commune	code_postal	type_local	surface_reelle_bati	nombre_pieces_principales	longitude	latitude	prix_m2	
1	2025-01-03	Vente	304000.0	Élancourt	78990.0	Maison	99.0		5.0	1.956081	48.774913	3070.71
4	2025-01-02	Vente	165300.0	Bois-d'Arcy	78390.0	Appartement	65.0		3.0	2.019029	48.799756	2543.08
7	2025-01-06	Vente	210000.0	Montigny-le Bretonneux	78180.0	Appartement	51.0		2.0	2.037769	48.773776	4117.65
11	2025-01-06	Vente	990650.0	Sonchamp	78120.0	Maison	312.0		6.0	1.906290	48.572459	3175.16
22	2025-01-06	Vente	990650.0	Sonchamp	78120.0	Maison	84.0		3.0	1.907317	48.571603	11793.45

```
df[["valeur_fonciere", "surface_reelle_bati", "prix_m2", "nombre_pieces_principales"]].describe()
```

[5] ✓ 0.0s Python

	valeur_fonciere	surface_reelle_bati	prix_m2	nombre_pieces_principales
count	1.612600e+04	16126.000000	16126.000000	16126.000000
mean	3.943886e+05	84.093452	5053.174630	3.699119
std	4.213317e+05	44.658399	5484.112544	1.619113
min	2.200000e+04	6.000000	1114.860000	0.000000
25%	2.000000e+05	55.000000	3028.767500	3.000000
50%	2.950000e+05	75.000000	3953.710000	4.000000
75%	4.474950e+05	103.000000	5316.127500	5.000000
max	9.010402e+06	625.000000	72973.330000	16.000000

```
df[["surface_reelle_bati", "valeur_fonciere", "prix_m2", "nombre_pieces_principales"]].corr()
```

a cellule (^Enter) ; Python

	surface_reelle_bati	valeur_fonciere	prix_m2	nombre_pieces_principales
surface_reelle_bati	1.000000	0.442999	-0.124748	0.844988
valeur_fonciere	0.442999	1.000000	0.701726	0.361628
prix_m2	-0.124748	0.701726	1.000000	-0.131412
nombre_pieces_principales	0.844988	0.361628	-0.131412	1.000000

```
top_communes = df["nom_commune"].value_counts().head(10).index
prix_par_commune = (
    df[df["nom_commune"].isin(top_communes)]
    .groupby("nom_commune")["prix_m2"]
    .mean()
    .sort_values(ascending=False)
)
prix_par_commune
```

[7] ✓ 0.0s

nom_commune	
Versailles	8163.679066
Saint-Germain-en-Laye	7573.954747
Chatou	6186.951432
Houilles	5274.861169
Sartrouville	5082.886623
Poissy	4536.764731
Conflans-Sainte-Honorine	4064.810226
Rambouillet	3897.013324
Plaisir	3525.236412
Élancourt	3058.969022

Name: prix_m2, dtype: float64